

```
// ===== lib/main.dart =====

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app.dart';
import 'src/core/app_mode.dart';
import 'src/core/rust_init.dart';
import 'src/core/watch_fit.dart';
import 'src/player/listen_stats.dart';
import 'src/player/watch_audio_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await WatchScreenShape.probe();
  final appMode = await readAppMode();
  final linkMode = appMode == appModeLink;
  AppModeNotifier.seed(appMode);
  final container = ProviderContainer();
  if (!linkMode) {
    container.read(rustInitProvider);
    container.read(listenStatsProvider);
    initWatchAudioService();
  }
  runApp(
    UncontrolledProviderScope(
      container: container,
      child: const XianYuWatchApp(),
    ),
  );
}

// ===== lib/app.dart =====

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'src/core/ambient.dart';
import 'src/core/app_mode.dart';
import 'src/core/keep_alive.dart' show LinkKeepAlive;
import 'src/core/settings.dart';
import 'src/core/watch_fit.dart';
import 'src/auth/auth_provider.dart';
import 'src/link/link_provider.dart';
import 'src/sync/sync_provider.dart';
import 'src/ui/local/local_music_hub.dart';
import 'src/ui/link/linkage_home.dart';
import 'src/update/app_update.dart';
```

```

class XianYuWatchApp extends ConsumerStatefulWidget {
  const XianYuWatchApp({super.key});

  @override
  ConsumerState<XianYuWatchApp> createState() => _XianYuWatchAppState();
}

class _XianYuWatchAppState extends ConsumerState<XianYuWatchApp> {
  final GlobalKey<NavigatorState> _navKey = GlobalKey<NavigatorState>();
  ProviderSubscription<String>? _modeSub;
  ProviderSubscription<AsyncValue<AppSettings>>? _settingsSub;

  @override
  void initState() {
    super.initState();
    _modeSub = ref.listenManual(appModeProvider, (prev, next) {
      if (prev != null && prev != next) {
        WidgetsBinding.instance.addPostFrameCallback((_) {
          _navKey.currentState?.popUntil((route) => route.isFirst);
        });
      }
    });
    Future.microtask(() {
      ref.read(syncProvider.notifier);
      ref.read(linkControllerProvider.notifier).init();
      ref.read(authProvider.notifier).init();
    });
    _startupUpdateCheck();
    initAmbientListener(ref);
    _settingsSub = ref.listenManual(settingsProvider, (prev, next) {
      _applyKeepScreenOn(next.valueOrNull?.keepScreenOn ?? true);
    });
  }

  @override
  void dispose() {
    _modeSub?.close();
    _settingsSub?.close();
    super.dispose();
  }

  Future<void> _startupUpdateCheck() async {
    await Future<void>.delayed(const Duration(seconds: 3));
    final latest = await ref.read(authProvider.notifier).fetchServerUpdate();
    if (latest == null || !hasNewVersion(latest)) return;
    if (!await claimUpdatePrompt()) return;
    final ctx = _navKey.currentState?.context;
    if (ctx == null || !ctx.mounted) return;
    await showUpdatePage(ctx, latest,
      onUpdate: () => toastUpdateOnPhone(ctx));
  }
}

```



```

    ]),
    child: nav,
  );
}
return Stack(
  children: [
    Positioned.fill(child: nav),
    _EdgeBackStrip(navigatorKey: _navKey),
    const Positioned.fill(child: _PairRequestHost()),
  ],
);
},
),
home: Consumer(
  builder: (context, ref, _) {
    final mode = ref.watch(appModeProvider);
    if (mode == appModeLink) LinkKeepAlive.start();
    return mode == appModeLink
      ? const LinkageHome()
      : const LocalMusicHub();
  },
),
);
}
}

```

```

class _EdgeBackStrip extends StatefulWidget {
  const _EdgeBackStrip({required this.navigatorKey});

  final GlobalKey<NavigatorState> navigatorKey;

  @override
  State<_EdgeBackStrip> createState() => _EdgeBackStripState();
}

```

```

class _EdgeBackStripState extends State<_EdgeBackStrip> {
  double _dx = 0;

  @override
  Widget build(BuildContext context) {
    final s = context.watchScale();
    return Positioned(
      left: 0,
      top: 0,
      bottom: 0,
      width: 26 * s,
      child: GestureDetector(
        behavior: HitTestBehavior.translucent,
        onHorizontalDragStart: (_) => _dx = 0,
        onHorizontalDragUpdate: (d) => _dx += d.delta.dx,
      ),
    );
  }
}

```

```

        onHorizontalDragEnd: (d) {
            final velocity = d.primaryVelocity ?? 0;
            final isBack = velocity > 300 || _dx >= 60 * s;
            _dx = 0;
            if (!isBack) return;
            widget.navigatorKey.currentState?.maybePop();
        },
    ),
);
}
}

class _PairRequestHost extends ConsumerStatefulWidget {
    const _PairRequestHost();

    @override
    ConsumerState<_PairRequestHost> createState() => _PairRequestHostState();
}

class _PairRequestHostState extends ConsumerState<_PairRequestHost> {
    final ScrollController _scroll = ScrollController();

    @override
    void dispose() {
        _scroll.dispose();
        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        final s = context.watchScale();
        final name = ref.watch(
            linkControllerProvider.select((st) => st.incomingName),
        );
        if (name.isEmpty) return const SizedBox.shrink();
        final vh = MediaQuery.of(context).size.height;
        return Container(
            color: const Color(0xFF0C0C0F),
            child: NotificationListener<ScrollEndNotification>({
                onNotification: (n) {
                    final m = n.metrics;
                    if (!m.hasContentDimensions || m.maxScrollExtent <= 0) return false;
                    final target =
                        (m.pixels / m.maxScrollExtent).round() * m.maxScrollExtent;
                    if ((target - m.pixels).abs() > 0.5) {
                        _scroll.animateTo(
                            target,
                            duration: const Duration(milliseconds: 160),
                            curve: Curves.easeOutCubic,
                        );
                    }
                }
            })
        );
    }
}

```

```

    }
    return false;
  },
  child: SingleChildScrollView(
    controller: _scroll,
    padding: EdgeInsets.symmetric(horizontal: 30 * s, vertical: 14 * s),
    child: ConstrainedBox(
      constraints: BoxConstraints(minHeight: vh - 28 * s),
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(
              Icons.watch_rounded,
              size: 44 * s,
              color: const Color(0xFF4A90D9),
            ),
            SizedBox(height: 12 * s),
            Text(
              '配对请求',
              style: TextStyle(
                fontSize: 18 * s,
                fontWeight: FontWeight.bold,
              ),
            ),
            SizedBox(height: 8 * s),
            Text(
              '「${name.isEmpty ? '手机' : name}」请求连接腕上弦予',
              textAlign: TextAlign.center,
              style: TextStyle(
                fontSize: 13 * s,
                height: 1.35,
                color: Colors.white.withValues(alpha: 0.6),
              ),
            ),
            SizedBox(height: 20 * s),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              crossAxisAlignment: CrossAxisAlignment.min,
              children: [
                FilledButton(
                  onPressed: () => ref
                    .read(linkControllerProvider.notifier)
                    .acceptIncoming(),
                  style: FilledButton.styleFrom(
                    backgroundColor: const Color(0xFFFF4D6E),
                    padding: EdgeInsets.symmetric(
                      horizontal: 18 * s,
                      vertical: 5 * s,
                    ),
                  ),

```

```

        minimumSize: Size(0, 36 * s),
        textStyle: TextStyle(fontSize: 13.5 * s),
      ),
      child: const Text('允许'),
    ),
    SizedBox(width: 12 * s),
    OutlinedButton(
      onPressed: () => ref
        .read(linkControllerProvider.notifier)
        .rejectIncoming(),
      style: OutlinedButton.styleFrom(
        padding: EdgeInsets.symmetric(
          horizontal: 18 * s,
          vertical: 5 * s,
        ),
        minimumSize: Size(0, 36 * s),
        textStyle: TextStyle(fontSize: 13.5 * s),
      ),
      child: const Text('拒绝'),
    ),
  ],
),
],
),
),
),
),
),
);
}
}
// ===== lib/src/link/protocol.dart =====
library;

import 'dart:convert';
import 'dart:typed_data';

const int kLinkProtocolVersion = 1;

const String kWatchLinkServiceUuid = 'f7a24b6c-9d3e-4f8a-b1c2-2e5d8a7f6b3a';

final Uint8List kFrameMagic = Uint8List.fromList([0x58, 0x59, 0x57, 0x31]);

const int kFrameHeaderBytes = 14;

const int kMaxPayloadBytes = 4 * 1024;

class LinkMsgType {
  static const int hello = 0x01;
  static const int bye = 0x02;

```

```
static const int ping = 0x03;
static const int pong = 0x04;

static const int state = 0x10;
static const int nowPlaying = 0x11;
static const int position = 0x12;

static const int lyric = 0x13;

static const int precache = 0x14;

static const int cmd = 0x20;

static const int backupFile = 0x21;

static const int backupAck = 0x22;

static const int watchLogFile = 0x23;

static const int effects = 0x24;

static const int chunk = 0x30;

static const int cloudBind = 0x41;

static bool known(int t) =>
    t == hello ||
    t == bye ||
    t == ping ||
    t == pong ||
    t == state ||
    t == nowPlaying ||
    t == position ||
    t == lyric ||
    t == precache ||
    t == cmd ||
    t == backupFile ||
    t == backupAck ||
    t == watchLogFile ||
    t == effects ||
    t == chunk ||
    t == cloudBind;
}

enum LinkPlayMode { order, shuffle, one }

LinkPlayMode linkPlayModeFromString(String? s) {
    switch (s) {
        case 'shuffle':
            return LinkPlayMode.shuffle;
    }
}
```

```

        case 'one':
            return LinkPlayMode.one;
        default:
            return LinkPlayMode.order;
    }
}

```

```

String linkPlayModeToString(LinkPlayMode m) {
    switch (m) {
        case LinkPlayMode.shuffle:
            return 'shuffle';
        case LinkPlayMode.one:
            return 'one';
        case LinkPlayMode.order:
            return 'order';
    }
}

```

```

class LinkCmdAction {
    static const String toggle = 'toggle';
    static const String next = 'next';
    static const String prev = 'prev';
    static const String like = 'like';
    static const String dislike = 'dislike';
    static const String mode = 'mode';
    static const String seek = 'seek';
    static const String volume = 'volume';

    static const String fx = 'fx';
}

```

```

class LinkMessage {
    LinkMessage(this.type, this.payload);

    final int type;
    final Map<String, dynamic> payload;

    String? action() => payload['action'] as String?;

    static LinkMessage hello({
        required int ver,
        required String role,
        String name = '',
    }) =>
        LinkMessage(LinkMsgType.hello, {'ver': ver, 'role': role, 'name': name});

    static LinkMessage state({
        required bool isPlaying,
        required LinkPlayMode playMode,
        required bool liked,
    }) =>
        LinkMessage(LinkMsgType.state, {
            'isPlaying': isPlaying,
            'playMode': playMode,
            'liked': liked,
        });
}

```

```

        double? volume,
        String? mvPhase,
    )) => LinkMessage(LinkMsgType.state, {
        'isplaying': isPlaying,
        'playMode': linkPlayModeToString(playMode),
        'liked': liked,
        'volume': ?volume,
        'mvPhase': ?mvPhase,
    });

    static LinkMessage nowPlaying({
        required String id,
        required String title,
        required String artist,
        required String album,
        String? cover,
        String? coverData,
        required double duration,
        bool daily = false,
    )) => LinkMessage(LinkMsgType.nowPlaying, {
        'id': id,
        'title': title,
        'artist': artist,
        'album': album,
        'cover': cover,
        'coverData': ?coverData,
        'duration': duration,
        'daily': daily,
    });

    static LinkMessage position({
        required double pos,
        required double duration,
    )) => LinkMessage(LinkMsgType.position, {'pos': pos, 'duration': duration});

    static LinkMessage lyric({required String id, required String payload}) =>
        LinkMessage(LinkMsgType.lyric, {
            'lyric': {'id': id, 'payload': payload},
        });

    static LinkMessage precache({
        required String id,
        String? coverData,
        String? lyricPayload,
    )) => LinkMessage(LinkMsgType.precache, {
        'precache': {'id': id, 'cover': ?coverData, 'lyric': ?lyricPayload},
    });

    static LinkMessage cmd(String action, [Map<String, dynamic>? arg]) =>
        LinkMessage(LinkMsgType.cmd, {'action': action, 'arg': ?arg});

```

```

static LinkMessage effects({required Map<String, dynamic> fx}) =>
    LinkMessage(LinkMsgType.effects, {'fx': fx});

static LinkMessage backupFile({
    required String name,
    required String content,
}) => LinkMessage(LinkMsgType.backupFile, {
    'name': name,
    'size': utf8.encode(content).length,
    'backup': content,
});

static LinkMessage backupAck({
    required String result,
}) => LinkMessage(LinkMsgType.backupAck, {'result': result});

static LinkMessage watchLogFile({
    required String name,
    required String content,
}) => LinkMessage(LinkMsgType.watchLogFile, {
    'name': name,
    'size': utf8.encode(content).length,
    'log': content,
});

static LinkMessage cloudBind({required String key, String? url}) =>
    LinkMessage(LinkMsgType.cloudBind, {
        'cloud_bind': {'key': key, 'url': ?url},
    });

Map<String, dynamic> toJson() => {'type': type, 'payload': payload};

@override
String toString() =>
    'LinkMessage(type=0x${type.toRadixString(16)}, payload=$payload)';
}

int crc16CcittFalse(List<int> bytes) {
    var crc = 0xFFFF;
    for (final b in bytes) {
        crc ^= b << 8;
        for (var i = 0; i < 8; i++) {
            crc = (crc & 0x8000) != 0
                ? ((crc << 1) ^ 0x1021) & 0xFFFF
                : (crc << 1) & 0xFFFF;
        }
    }
    return crc;
}

```

```
int _readU16(UInt8List b, int o) => (b[o] << 8) | b[o + 1];
```

```
int _readU32(UInt8List b, int o) =>
    (b[o] << 24) | (b[o + 1] << 16) | (b[o + 2] << 8) | b[o + 3];
```

```
void _writeU16(UInt8List b, int o, int v) {
    b[o] = (v >> 8) & 0xFF;
    b[o + 1] = v & 0xFF;
}
```

```
void _writeU32(UInt8List b, int o, int v) {
    b[o] = (v >> 24) & 0xFF;
    b[o + 1] = (v >> 16) & 0xFF;
    b[o + 2] = (v >> 8) & 0xFF;
    b[o + 3] = v & 0xFF;
}
```

```
List<UInt8List> encodeFrames(
    LinkMessage msg, {
        required int Function() nextSeq,
    }) {
    final jsonBytes = utf8.encode(jsonEncode(msg.payload));

    if (jsonBytes.length <= kMaxPayloadBytes) {
        return [_encodeOne(msg.type, nextSeq(), jsonBytes)];
    }

    var chunkSize = kMaxPayloadBytes - 512;
    var chunks = _splitUtf8Safe(jsonBytes, chunkSize);
    while (_chunkFramesOverflow(chunks)) {
        chunkSize = chunkSize ~/ 2;
        chunks = _splitUtf8Safe(jsonBytes, chunkSize);
    }

    final cid = DateTime.now().microsecondsSinceEpoch & 0xFFFFFFFF;
    final frames = <UInt8List>[];
    final seqBase = nextSeq();
    for (var i = 0; i < chunks.length; i++) {
        frames.add(
            _encodeOne(
                LinkMsgType.chunk,
                (seqBase + i) & 0xFFFF,
                utf8.encode(
                    _chunkPayloadJson(chunks[i], cid, chunks.length, i, msg.type),
                ),
            ),
        );
    }
    return frames;
}
```

```

String _chunkPayloadJson(
    UInt8List chunk,
    int cid,
    int total,
    int idx,
    int type,
) => jsonEncode({
    'cid': cid,
    'total': total,
    'idx': idx,
    'type': type,
    'data': utf8.decode(chunk, allowMalformed: false),
});

bool _chunkFramesOverflow(List<UInt8List> chunks) => chunks.any(
    (c) =>
        utf8.encode(_chunkPayloadJson(c, 0, chunks.length, 0, 0)).length >
            kMaxPayloadBytes,
);

UInt8List _encodeOne(int type, int seq, List<int> jsonBytes) {
    if (jsonBytes.length > kMaxPayloadBytes) {
        throw ArgumentError('payload too large: ${jsonBytes.length}');
    }
    final frame = UInt8List(kFrameHeaderBytes + jsonBytes.length);
    frame.setRange(0, 4, kFrameMagic);
    frame[4] = kLinkProtocolVersion;
    frame[5] = type;
    _writeU16(frame, 6, seq & 0xFFFF);
    _writeU32(frame, 8, jsonBytes.length);
    frame.setRange(kFrameHeaderBytes, frame.length, jsonBytes);
    final crcInput = frame.sublist(4);
    _writeU16(frame, 12, crc16CcittFalse(crcInput));
    return frame;
}

List<UInt8List> _splitUtf8Safe(UInt8List bytes, int maxChunk) {
    final out = <UInt8List>[];
    var start = 0;
    while (start < bytes.length) {
        var end = start + maxChunk < bytes.length ? start + maxChunk : bytes.length;
        if (end < bytes.length) {
            while (end > start && (bytes[end] & 0xC0) == 0x80) {
                end--;
            }
        }
        out.add(UInt8List.sublistView(bytes, start, end));
        start = end;
    }
}

```

```

        return out;
    }

class FrameDecoder {
    final _buf = ByteBuffer();
    final _chunks = <int, _ChunkSession>{};

    List<LinkMessage> feed(List<int> bytes) {
        _buf.add(bytes);
        final out = <LinkMessage>[];
        while (true) {
            final data = _buf.asBytes();
            var magicIdx = _findMagic(data);
            if (magicIdx < 0) {
                final keep = data.length < 4 ? data.length : 3;
                _buf.clear();
                if (keep > 0) _buf.add(data.sublist(data.length - keep));
                break;
            }
            if (magicIdx > 0) {
                final rest = data.sublist(magicIdx);
                _buf.clear();
                _buf.add(rest);
                continue;
            }
            if (data.length < kFrameHeaderBytes) break;
            final len = _readU32(data, 8);
            if (len > kMaxPayloadBytes) {
                _buf.clear();
                _buf.add(data.sublist(4));
                continue;
            }
            if (data.length < kFrameHeaderBytes + len) break;
            final frame = data.sublist(0, kFrameHeaderBytes + len);
            _buf.clear();
            _buf.add(data.sublist(kFrameHeaderBytes + len));

            final msg = _decodeFrame(frame);
            if (msg != null) out.add(msg);
        }
        return out;
    }

    LinkMessage? _decodeFrame(Uint8List frame) {
        try {
            return _decodeFrameInner(frame);
        } catch (_) {
            return null;
        }
    }
}

```

```

LinkMessage? _decodeFrameInner(UInt8List frame) {
    final type = frame[5];
    final len = _readU32(frame, 8);
    final expectCrc = _readU16(frame, 12);
    final check = UInt8List.fromList(frame.sublist(4, kFrameHeaderBytes + len));
    check[8] = 0;
    check[9] = 0;
    final actualCrc = crc16CcittFalse(check);
    if (expectCrc != actualCrc) return null;
    if (!LinkMsgType.known(type)) return null;
    final payload = utf8.decode(
        frame.sublist(kFrameHeaderBytes, kFrameHeaderBytes + len),
        allowMalformed: false,
    );
    if (type == LinkMsgType.chunk) return _reassembleChunk(payload);
    return LinkMessage(type, jsonDecode(payload) as Map<String, dynamic>);
}

```

```

LinkMessage? _reassembleChunk(String payload) {
    final map = jsonDecode(payload) as Map<String, dynamic>;
    final cid = map['cid'] as int;
    final total = map['total'] as int;
    final idx = map['idx'] as int;
    final innerType = map['type'] as int;
    final data = map['data'] as String;
    if (total <= 0 || total > 8192) return null;
    if (idx < 0 || idx >= total) return null;
    if (_chunks.length > 8 && !_chunks.containsKey(cid)) {
        _chunks.remove(_chunks.keys.first);
    }
    final session = _chunks.putIfAbsent(
        cid,
        () => _ChunkSession(total, innerType),
    );
    session.parts[idx] = data;
    if (session.parts.length == total) {
        _chunks.remove(cid);
        final joined = session.parts.keys.toList()..sort();
        final full = joined.map((i) => session.parts[i!]).join();
        return LinkMessage(
            session.innerType,
            jsonDecode(full) as Map<String, dynamic>,
        );
    }
    return null;
}

```

```

static int _findMagic(UInt8List d) {
    outer:

```

```

    for (var i = 0; i + 4 <= d.length; i++) {
        for (var j = 0; j < 4; j++) {
            if (d[i + j] != kFrameMagic[j]) continue outer;
        }
        return i;
    }
    return -1;
}
}

class _ChunkSession {
    _ChunkSession(this.total, this.innerType);
    final int total;
    final int innerType;
    final Map<int, String> parts = {};
}

class ByteBuffer {
    final List<int> _data = [];

    void add(List<int> bytes) => _data.addAll(bytes);

    Uint8List asBytes() => Uint8List.fromList(_data);

    void clear() => _data.clear();

    int get length => _data.length;
}

int Function() makeSeqGenerator() {
    var seq = 0;
    return () {
        seq = (seq + 1) & 0xFFFF;
        return seq;
    };
}

// ===== lib/src/link/rfcomm_client.dart =====
import 'dart:async';

import 'package:flutter/services.dart';

class LinkConnectionEvent {
    const LinkConnectionEvent({required this.connected, required this.name});

    final bool connected;
    final String name;
}

class BondedDevice {
    const BondedDevice({required this.address, required this.name});

```

```

    final String address;
    final String name;

    static BondedDevice fromMap(Object? m) {
        final map = m as Map;
        return BondedDevice(
            address: map['address'] as String? ?? '',
            name: map['name'] as String? ?? '',
        );
    }
}

class IncomingPairRequest {
    const IncomingPairRequest({required this.name, required this.address});

    final String name;
    final String address;

    static IncomingPairRequest fromMap(Object? m) {
        final map = m as Map? ?? const {};
        return IncomingPairRequest(
            name: (map['name'] as String?) ?? '',
            address: (map['address'] as String?) ?? '',
        );
    }
}

class LinkClientChannel {
    static const MethodChannel _ch = MethodChannel('xianyu/watch_link');

    final _rawCtrl = StreamController<Uint8List>.broadcast();
    final _connCtrl = StreamController<LinkConnectionEvent>.broadcast();
    final _permCtrl = StreamController<bool>.broadcast();
    final _incomingCtrl = StreamController<IncomingPairRequest>.broadcast();
    bool _bound = false;

    Stream<Uint8List> get onRaw => _rawCtrl.stream;

    Stream<LinkConnectionEvent> get onConnection => _connCtrl.stream;

    Stream<bool> get onPermission => _permCtrl.stream;

    Stream<IncomingPairRequest> get onIncomingPair => _incomingCtrl.stream;

    void bind() {
        if (_bound) return;
        _bound = true;
        _ch.setMethodCallHandler((call) async {
            switch (call.method) {

```

```

        case 'onRaw':
            final bytes = call.arguments as Uint8List?;
            if (bytes != null && bytes.isNotEmpty) _rawCtrl.add(bytes);
        case 'onConnection':
            final args = call.arguments as Map?;
            _connCtrl.add(LinkConnectionEvent(
                connected: args?['connected'] == true,
                name: (args?['name'] as String?) ?? '',
            ));
        case 'onPermission':
            _permCtrl.add(call.arguments == true);
        case 'onIncomingPair':
            final req = IncomingPairRequest.fromMap(call.arguments);
            if (req.address.isNotEmpty) _incomingCtrl.add(req);
    }
});
}

```

```

Future<bool> sppSupported() async {
    try {
        return await _ch.invokeMethod('sppSupported') == true;
    } catch (_) {
        return false;
    }
}

```

```

Future<List<BondedDevice>> pairedDevices() async {
    try {
        final list = await _ch.invokeMethod<List<dynamic>>('pairedDevices');
        return (list ?? const []).map(BondedDevice.fromMap).toList();
    } catch (_) {
        return const [];
    }
}

```

```

Future<void> connect(String address) async {
    try {
        await _ch.invokeMethod('connect', {'address': address});
    } catch (_) {}
}

```

```

Future<void> disconnect() async {
    try {
        await _ch.invokeMethod('disconnect');
    } catch (_) {}
}

```

```

Future<void> startServer() async {
    try {
        await _ch.invokeMethod('startServer');
    }
}

```

```
    } catch (_) {}
  }

Future<void> acceptPair() async {
  try {
    await _ch.invokeMethod('acceptPair');
  } catch (_) {}
}

Future<void> rejectPair() async {
  try {
    await _ch.invokeMethod('rejectPair');
  } catch (_) {}
}

Future<void> send(Uint8List bytes) async {
  try {
    await _ch.invokeMethod('send', {'bytes': bytes});
  } catch (_) {}
}

Future<bool> hasPermission() async {
  try {
    return await _ch.invokeMethod('hasPermission') == true;
  } catch (_) {
    return false;
  }
}

Future<void> requestPermission() async {
  try {
    await _ch.invokeMethod('requestPermission');
  } catch (_) {}
}

Future<void> notifyNowPlaying(String title, String artist) async {
  try {
    await _ch.invokeMethod('notifyNowPlaying', {
      'title': title,
      'artist': artist,
    });
  } catch (_) {}
}
}

// ===== lib/src/link/link_provider.dart =====
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:typed_data';
```

```
import 'package:flutter/widgets.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:crypto/crypto.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../core/ambient.dart';
import '../effects/sound_effect_provider.dart';
import 'cloud_client.dart';
import 'protocol.dart';
import 'rfcomm_client.dart';

const String kDefaultCloudRelayUrl = 'wss://api.xianyumusic.cn/watch-relay';

enum LinkPhase { disconnected, connecting, connected }

class LinkNowPlaying {
  const LinkNowPlaying({
    required this.id,
    required this.title,
    required this.artist,
    required this.album,
    this.cover,
    required this.duration,
    this.daily = false,
  });

  final String id;
  final String title;
  final String artist;
  final String album;
  final String? cover;
  final double duration;

  final bool daily;

  bool get coverIsFile =>
    cover != null && cover!.isNotEmpty && !cover!.startsWith('http');

  static LinkNowPlaying fromPayload(Map<String, dynamic> p) => LinkNowPlaying(
    id: p['id'] as String? ?? '',
    title: p['title'] as String? ?? '',
    artist: p['artist'] as String? ?? '',
    album: p['album'] as String? ?? '',
    cover: p['cover'] as String?,
    duration: (p['duration'] as num?)?.toDouble() ?? 0,
    daily: p['daily'] as bool? ?? false,
  );

  LinkNowPlaying copyWith({String? cover}) => LinkNowPlaying(
    id: id,
```

```
        title: title,
        artist: artist,
        album: album,
        cover: cover ?? this.cover,
        duration: duration,
        daily: daily,
    );
}

class LinkState {
    final LinkPhase phase;
    final String phoneName;

    final bool autoEnabled;

    final String? pairedAddress;
    final String? pairedName;

    final LinkNowPlaying? now;
    final bool isPlaying;
    final LinkPlayMode playMode;
    final bool liked;
    final double? volume;

    final double position;

    final bool viaCloud;

    final String? lyricSongId;
    final String? lyricPayload;

    final String incomingName;
    final String incomingAddress;

    final String? mvPhase;

    const LinkState({
        this.phase = LinkPhase.disconnected,
        this.phoneName = '',
        this.autoEnabled = true,
        this.pairedAddress,
        this.pairedName,
        this.now,
        this.isPlaying = false,
        this.playMode = LinkPlayMode.order,
        this.liked = false,
        this.volume,
        this.position = 0,
        this.viaCloud = false,
        this.lyricSongId,
```

```

        this.lyricPayload,
        this.incomingName = '',
        this.incomingAddress = '',
        this.mvPhase,
    ));

```

```

LinkState copyWith({
    LinkPhase? phase,
    String? phoneName,
    bool? autoEnabled,
    Object? pairedAddress = _noChange,
    Object? pairedName = _noChange,
    Object? now = _noChange,
    bool? isPlaying,
    LinkPlayMode? playMode,
    bool? liked,
    Object? volume = _noChange,
    double? position,
    bool? viaCloud,
    Object? lyricSongId = _noChange,
    Object? lyricPayload = _noChange,
    String? incomingName,
    String? incomingAddress,
    Object? mvPhase = _noChange,
}) => LinkState(
    phase: phase ?? this.phase,
    phoneName: phoneName ?? this.phoneName,
    autoEnabled: autoEnabled ?? this.autoEnabled,
    pairedAddress: pairedAddress == _noChange
        ? this.pairedAddress
        : pairedAddress as String?,
    pairedName: pairedName == _noChange
        ? this.pairedName
        : pairedName as String?,
    now: now == _noChange ? this.now : now as LinkNowPlaying?,
    isPlaying: isPlaying ?? this.isPlaying,
    playMode: playMode ?? this.playMode,
    liked: liked ?? this.liked,
    volume: volume == _noChange ? this.volume : volume as double?,
    position: position ?? this.position,
    viaCloud: viaCloud ?? this.viaCloud,
    lyricSongId: lyricSongId == _noChange
        ? this.lyricSongId
        : lyricSongId as String?,
    lyricPayload: lyricPayload == _noChange
        ? this.lyricPayload
        : lyricPayload as String?,
    incomingName: incomingName ?? this.incomingName,
    incomingAddress: incomingAddress ?? this.incomingAddress,
    mvPhase: mvPhase == _noChange ? this.mvPhase : mvPhase as String?,

```

```

    );
}

const Object _noChange = Object();

class LinkController extends StateNotifier<LinkState>
    implements SoundEffectLinkHost {
    LinkController() : super(const LinkState());

    final LinkClientChannel _channel = LinkClientChannel();
    final CloudLinkClient _cloud = CloudLinkClient();
    FrameDecoder _decoder = FrameDecoder();
    final int Function() _nextSeq = makeSeqGenerator();

    static const _heartbeatInterval = Duration(seconds: 3);
    static const _deadAfter = Duration(seconds: 10);
    static const _minBackoff = Duration(seconds: 1);

    static const _maxBackoff = Duration(seconds: 15);

    final List<StreamSubscription<dynamic>> _subs = [];
    Timer? _heartbeat;
    Timer? _interpolate;
    Timer? _reconnect;
    Duration _backoff = _minBackoff;

    DateTime _lastFrame = DateTime.fromMillisecondsSinceEpoch(0);

    double _posBase = 0;

    DateTime _posAt = DateTime.fromMillisecondsSinceEpoch(0);

    bool _viaCloud = false;

    bool _sppAvailable = false;

    String _cloudKey = '';
    String _cloudUrl = '';

    bool _cloudTryActive = false;

    bool _disposed = false;

    final Map<String, String> _lyricCache = {};

    Completer<String>? _backupAckCompleter;

    SoundEffectManager? _fxMgr;

    void attachFx(SoundEffectManager mgr) {

```

```

    _fxMgr = mgr;
    _syncFxMirror();
}

@Override
set state(LinkState value) {
    final prevPhase = state.phase;
    super.state = value;
    if (prevPhase != value.phase) _syncFxMirror();
}

void _syncFxMirror() {
    if (state.phase == LinkPhase.connected) {
        _fxMgr?.linkRemote(true, host: this);
    } else {
        _fxMgr?.linkRemote(false);
    }
}

@Override
bool get fxActive => state.phase == LinkPhase.connected;

@Override
void sendFx(SoundEffectSettings s) {
    _sendCmd(LinkCmdAction.fx, arg: s.toJson());
}

double? get linkedSpeed {
    final mgr = _fxMgr;
    if (mgr == null) return null;
    return mgr.state.settings.playbackRate / 100.0;
}

void setLinkedSpeed(double v) {
    final mgr = _fxMgr;
    if (mgr == null || state.phase != LinkPhase.connected) return;
    final rate = (v * 100).clamp(50.0, 200.0);
    mgr.set(mgr.state.settings.copyWith(playbackRate: rate));
}

void Function(String title, String artist)? onBackgroundNowPlaying;

bool Function()? isAmbient;

Future<void> init() async {
    _channel.bind();
    _sppAvailable =
        Platform.isAndroid || await _channel.sppSupported();
    onBackgroundNowPlaying = (title, artist) {
        _channel.notifyNowPlaying(title, artist);
    }
}

```

```

    };
    _subs.add(_channel.onRaw.listen(_onRaw));
    _subs.add(_channel.onConnection.listen(_onConnection));
    _subs.add(_channel.onPermission.listen(_onPermission));
    _subs.add(_channel.onIncomingPair.listen(_onIncomingPair));
    _subs.add(_cloud.onRaw.listen(_onRaw));
    _subs.add(_cloud.onEvent.listen(_onCloudEvent));
    _channel.startServer();

    final prefs = await SharedPreferences.getInstance();
    final addr = prefs.getString('watch.pairedAddress');
    final name = prefs.getString('watch.pairedName');
    final auto = prefs.getBool('watch.autoEnabled') ?? true;
    _cloudKey = prefs.getString('watch.cloudKey') ?? '';
    _cloudUrl = prefs.getString('watch.cloudUrl') ?? '';
    state = state.copyWith(
      autoEnabled: auto,
      pairedAddress: addr,
      pairedName: name,
    );
    if (addr != null && auto) {
      final linkageOn = prefs.getBool('watchLinkageEnabled') ?? true;
      if (linkageOn) _attemptConnect();
    }
  }

  @override
  void dispose() {
    _disposed = true;
    for (final s in _subs) {
      s.cancel();
    }
    _heartbeat?.cancel();
    _interpolate?.cancel();
    _reconnect?.cancel();
    _connectWatchdog?.cancel();
    _cloud.close();
    _channel.disconnect();
    super.dispose();
  }

  Future<void> selectDevice(BondedDevice dev) async {
    _reconnect?.cancel();
    _backoff = _minBackoff;
    state = state.copyWith(
      pairedAddress: dev.address,
      pairedName: dev.name,
      phase: LinkPhase.connecting,
      now: null,
      isPlaying: false,

```

```

        position: 0,
    );
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString('watch.pairedAddress', dev.address);
    await prefs.setString('watch.pairedName', dev.name);
    _attemptConnect();
}

```

```

Future<void> disconnectManually() async {
    _reconnect?.cancel();
    _stopAlive();
    _viaCloud = false;
    _cloudTryActive = false;
    state = state.copyWith(
        phase: LinkPhase.disconnected,
        now: null,
        viaCloud: false,
    );
    await _cloud.close();
    await _channel.disconnect();
}

```

```

Future<void> setAutoEnabled(bool v) async {
    state = state.copyWith(autoEnabled: v);
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('watch.autoEnabled', v);
    if (v) {
        final addr = state.pairedAddress;
        if (addr != null && state.phase == LinkPhase.disconnected) {
            _backoff = _minBackoff;
            _attemptConnect();
        } else if (_cloudKey.isNotEmpty &&
            state.phase == LinkPhase.disconnected) {
            _backoff = _minBackoff;
            _tryCloud();
        }
    } else {
        _reconnect?.cancel();
    }
}

```

```

void toggle() {
    if (state.phase != LinkPhase.connected) return;

    state = state.copyWith(isPlaying: !state.isPlaying);
    _sendCmd(LinkCmdAction.toggle);
}

void next() => _sendCmd(LinkCmdAction.next);
void prev() => _sendCmd(LinkCmdAction.prev);
void like() => _sendCmd(LinkCmdAction.like);

```

```
void dislike() => _sendCmd(LinkCmdAction.dislike);
void cycleMode() => _sendCmd(LinkCmdAction.mode);
void seek(double pos) => _sendCmd(LinkCmdAction.seek, arg: {'pos': pos});

void setVolume(double v) =>
    _sendCmd(LinkCmdAction.volume, arg: {'v': v.clamp(0.0, 1.0)});

void retry() {
    _reconnect?.cancel();
    _backoff = _minBackoff;
    _btPhaseStart = null;
    state = state.copyWith(phase: LinkPhase.disconnected);
    _attemptConnect();
}

Future<void> acceptIncoming() async {
    _reconnect?.cancel();
    _backoff = _minBackoff;
    final addr = state.incomingAddress;
    final name = state.incomingName;
    state = state.copyWith(
        phase: LinkPhase.connecting,
        pairedAddress: addr.isEmpty ? state.pairedAddress : addr,
        pairedName: name.isEmpty ? state.pairedName : name,
        incomingName: '',
        incomingAddress: '',
    );
    if (addr.isNotEmpty) {
        final prefs = await SharedPreferences.getInstance();
        await prefs.setString('watch.pairedAddress', addr);
        await prefs.setString('watch.pairedName', name);
    }
    _channel.acceptPair();
}

void rejectIncoming() {
    _channel.rejectPair();
    state = state.copyWith(incomingName: '', incomingAddress: '');
}

void _onIncomingPair(IncomingPairRequest req) {
    if (state.phase != LinkPhase.disconnected) {
        _channel.rejectPair();
        return;
    }
    state = state.copyWith(
        incomingName: req.name,
        incomingAddress: req.address,
    );
}
```

```
Future<List<BondedDevice>> loadPairedDevices() => _channel.pairedDevices();
```

```
Future<bool> hasBluetoothPermission() => _channel.hasPermission();
```

```
Future<void> requestBluetoothPermission() => _channel.requestPermission();
```

```
void _sendCmd(String action, {Map<String, dynamic>? arg}) {  
  if (state.phase != LinkPhase.connected) return;  
  _send(LinkMessage.cmd(action, arg));  
}
```

```
static const String backupPushSaved = 'saved';  
static const String backupPushCancelled = 'cancelled';  
static const String backupPushTimeout = 'timeout';  
static const String backupPushOffline = 'offline';
```

```
Future<String> pushBackup({  
  required String name,  
  required String content,  
  Duration timeout = const Duration(seconds: 30),  
}) async {  
  if (state.phase != LinkPhase.connected) return backupPushOffline;  
  final pending = Completer<String>();  
  _backupAckCompleter = pending;  
  try {  
    _send(LinkMessage.backupFile(name: name, content: content));  
    return await pending.future.timeout(timeout,  
      onTimeout: () => backupPushTimeout);  
  } finally {  
    _backupAckCompleter = null;  
  }  
}
```

```
Future<String> pushLogFile({  
  required String name,  
  required String content,  
  Duration timeout = const Duration(seconds: 30),  
}) async {  
  if (state.phase != LinkPhase.connected) return backupPushOffline;  
  final pending = Completer<String>();  
  _backupAckCompleter = pending;  
  try {  
    _send(LinkMessage.watchLogFile(name: name, content: content));  
    return await pending.future.timeout(timeout,  
      onTimeout: () => backupPushTimeout);  
  } finally {  
    _backupAckCompleter = null;  
  }  
}
```

```

void _attemptConnect() {
    if (state.phase == LinkPhase.connected ||
        state.phase == LinkPhase.connecting) {
        return;
    }

    if (!_sppAvailable) {
        if (_cloudKey.isNotEmpty) _tryCloud();
        return;
    }

    final addr = state.pairedAddress;
    if ((addr == null || addr.isEmpty)) {
        if (_cloudKey.isNotEmpty) _tryCloud();
        return;
    }

    _btPhaseStart ??= DateTime.now();
    _startBtRound(addr);
}

DateTime? _btPhaseStart;

static const _btPhaseBudget = Duration(seconds: 60);

void _startBtRound(String addr) {
    _wasExpectingConnect = true;
    state = state.copyWith(phase: LinkPhase.connecting);
    _channel.connect(addr);
    _connectWatchdog?.cancel();
    _connectWatchdog = Timer(const Duration(seconds: 15), _onBtRoundTimeout);
}

void _onBtRoundTimeout() {
    if (state.phase != LinkPhase.connecting || _viaCloud) return;
    final addr = state.pairedAddress;
    final start = _btPhaseStart;
    if (addr != null &&
        addr.isNotEmpty &&
        start != null &&
        DateTime.now().difference(start) < _btPhaseBudget) {
        _channel.disconnect();
        _startBtRound(addr);
        return;
    }

    _btPhaseStart = null;
    _wasExpectingConnect = false;
    _channel.disconnect();
    state = state.copyWith(
        phase: LinkPhase.disconnected,
        now: null,

```

```

        isPlaying: false,
        position: 0,
    );
    if (_cloudKey.isNotEmpty) {
        _tryCloud();
    } else {
        _scheduleReconnect();
    }
}

void _onConnection(LinkConnectionEvent evt) {
    _reconnect?.cancel();
    _connectWatchdog?.cancel();
    state = state.copyWith(incomingName: '', incomingAddress: '');
    if (evt.connected) {
        _onLinkUp(name: evt.name, viaCloud: false);
    } else {
        final wasConnected = state.phase == LinkPhase.connected;
        _stopAlive();
        state = state.copyWith(
            phase: LinkPhase.disconnected,
            now: null,
            isPlaying: false,
            position: 0,
        );

        final start = _btPhaseStart;
        final addr = state.pairedAddress;
        if (!wasConnected &&
            addr != null &&
            addr.isNotEmpty &&
            start != null &&
            DateTime.now().difference(start) < _btPhaseBudget) {
            _startBtRound(addr);
            return;
        }
        if ((wasConnected && state.autoEnabled) || _wasExpectingConnect) {
            if (_cloudKey.isNotEmpty) {
                _tryCloud();
            } else {
                _scheduleReconnect();
            }
        }
        _wasExpectingConnect = false;
    }
}

void _tryCloud() {
    if (_cloudKey.isEmpty) return;
    if (!state.autoEnabled) return;

```

..... (中间部分略)

```

        INNER JOIN songs s ON ph.song_id = s.id
        WHERE ph.played_at >= {} AND ph.event IN ('play', 'play_time') AND ph.song_id IS NOT NU
LL
        GROUP BY day_offset",
        start_time, day_seconds, start_time
    );

    let mut stmt = conn.prepare(&sql_activity).map_err(|e| e.to_string())?;
    let rows = stmt
        .query_map([], |row| Ok((row.get::<_, i64>(0)?, row.get::<_, i64>(1)?)))
        .map_err(|e| e.to_string())?;

    for row in rows.flatten() {
        let offset = row.0;
        if offset >= 0 && offset < 7 {
            recent_activity[offset as usize] = row.1;
        }
    }
}

Ok(BehaviorStats {
    total_plays,
    total_duration,
    top_songs,
    top_songs_by_duration,
    top_artists,
    top_albums,
    hour_distribution,
    recent_activity,
})
}

#[derive(Serialize)]
pub struct ListenDurations {
    pub daily: i64,
    pub weekly: i64,
    pub total: i64,
    pub today_play_count: i64,
}

pub fn get_listen_durations(conn: &rusqlite::Connection) -> Result<ListenDurations, String> {
    let daily_ms: i64 = conn
        .query_row(
            "SELECT COALESCE(play_time_ms, 0) FROM daily_stats
            WHERE date = strftime('%Y-%m-%d', 'now', 'localtime')",
            [],
            |row| row.get(0),
        )
        .unwrap_or(0);

    let weekly_ms: i64 = conn

```

```

        .query_row(
            "SELECT COALESCE(SUM(play_time_ms), 0) FROM daily_stats
            WHERE date >= strftime('%Y-%m-%d', 'now', 'localtime', '-6 days')",
            [],
            |row| row.get(0),
        )
        .unwrap_or(0);

let total_ms: i64 = conn
    .query_row(
        "SELECT COALESCE(total_play_time_ms, 0) FROM global_stats WHERE id = 1",
        [],
        |row| row.get(0),
    )
    .unwrap_or(0);

let today_count: i64 = conn
    .query_row(
        "SELECT COALESCE(play_count, 0) FROM daily_stats
        WHERE date = strftime('%Y-%m-%d', 'now', 'localtime')",
        [],
        |row| row.get(0),
    )
    .unwrap_or(0);

Ok(ListenDurations {
    daily: daily_ms / 1000,
    weekly: weekly_ms / 1000,
    total: total_ms / 1000,
    today_play_count: today_count,
})
}

#[cfg(test)]
mod playback_count_tests {
    use super::*;

    #[test]
    fn periodic_time_flushes_only_increment_play_count_once() {
        let conn = rusqlite::Connection::open_in_memory().expect("open in-memory database");
        crate::database::schema::ensure_base_schema(&conn).expect("create schema");
        let identity = PortableSongIdentity {
            title: "Demo".to_string(),
            artist: "Artist".to_string(),
            album: "Album".to_string(),
            duration_ms: 195_000,
            track_number: Some(1),
        };
        record_aggregate_play(&conn, &identity, 1_700_000_000, 30_000, false, false, true)
    }
}

```

```

        .expect("record initial play chunk");
    for offset in 1..=12 {
        record_aggregate_play(
            &conn,
            &identity,
            1_700_000_000 + offset,
            30_000,
            false,
            false,
            false,
        )
        .expect("record time-only chunk");
    }

    let (play_count, play_time_ms): (i64, i64) = conn
        .query_row(
            "SELECT total_play_count, total_play_time_ms FROM global_stats WHERE id = 1",
            [],
            |row| Ok((row.get(0)?, row.get(1)?)),
        )
        .expect("read global stats");
    assert_eq!(play_count, 1);
    assert_eq!(play_time_ms, 390_000);

    let recent_count: i64 = conn
        .query_row("SELECT COUNT(*) FROM recent_plays", [], |row| row.get(0))
        .expect("read recent plays");
    assert_eq!(recent_count, 1);
}

}

// ===== rust/src/toolbox.rs =====

use crate::music::tags::{
    extract_text_metadata, read_tagged_file_from_path, write_metadata_to_file, EmbedMetadataRequest,
};
use crate::music::utils::is_supported_library_extension;
use crate::security::path_validator;
use lofty::prelude::*;
use regex::Regex;
use serde::{Deserialize, Serialize};
use std::fs;
use std::path::{Path, PathBuf};
use std::sync::{Arc, Mutex, OnceLock};
use walkdir::WalkDir;

static TRACK_PREFIX_RE: OnceLock<Regex> = OnceLock::new();
static SOURCE_PREFIX_RE: OnceLock<Regex> = OnceLock::new();

#[derive(Debug, Serialize, Deserialize)]
pub struct RenameConfig {

```

```

    pub mode: String,
    pub template: String,
    pub remove_track_prefix: bool,
    pub remove_source_prefix: bool,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct RenamePreview {
    pub original_path: String,
    pub original_name: String,
    pub new_name: String,
    pub status: String,
    pub error: Option<String>,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct RenameOperation {
    pub original_path: String,
    pub new_name: String,
}

fn sanitize_filename(name: &str) -> String {
    let invalid_chars = ['<', '>', ':', '"', '/', '\\', '|', '?', '*'];
    let mut sanitized = String::new();
    for c in name.chars() {
        if invalid_chars.contains(&c) {
            sanitized.push('_');
        } else {
            sanitized.push(c);
        }
    }
    sanitized.trim().to_string()
}

fn process_file(path: &Path, config: &RenameConfig) -> RenamePreview {
    let original_name = path
        .file_name()
        .unwrap_or_default()
        .to_string_lossy()
        .to_string();
    let original_path_str = path.to_string_lossy().to_string();
    let ext = path
        .extension()
        .unwrap_or_default()
        .to_string_lossy()
        .to_string();

    if config.mode == "tags" || config.mode == "auto" {
        if let Ok(tagged_file) = read_tagged_file_from_path(path) {
            let metadata = extract_text_metadata(&tagged_file);

```

```

let title = metadata.title.unwrap_or_default();
let artist = metadata.artist.unwrap_or_default();
let album = metadata.album.unwrap_or_default();

let year = tagged_file
    .primary_tag()
    .and_then(|tag| tag.year())
    .map(|y| y.to_string())
    .unwrap_or_default();
let track = tagged_file
    .primary_tag()
    .and_then(|tag| tag.track())
    .map(|t| format!("{:02}", t))
    .unwrap_or_default();

if !title.is_empty() {
    let mut new_name_base = config.template.clone();
    new_name_base = new_name_base.replace("{title}", &title);
    new_name_base = new_name_base.replace("{artist}", &artist);
    new_name_base = new_name_base.replace("{album}", &album);
    new_name_base = new_name_base.replace("{year}", &year);
    new_name_base = new_name_base.replace("{track}", &track);

    let new_name = format!("{}", sanitize_filename(&new_name_base), ext);

    if new_name != original_name {
        return RenamePreview {
            original_path: original_path_str,
            original_name,
            new_name,
            status: "tags".to_string(),
            error: None,
        };
    } else if config.mode == "tags" {
        return RenamePreview {
            original_path: original_path_str,
            original_name: original_name.clone(),
            new_name: original_name,
            status: "skipped".to_string(),
            error: Some("Already named correctly".to_string()),
        };
    }
}

if config.mode == "tags" {
    return RenamePreview {
        original_path: original_path_str,
        original_name: original_name.clone(),
        new_name: original_name,
    }
}

```

```

        status: "skipped".to_string(),
        error: Some("Missing tags".to_string()),
    });
}
}

if config.mode == "rules" || config.mode == "auto" {
    let mut cleaned_name = original_name.clone();

    if let Some(stem) = path.file_stem() {
        let mut stem_str = stem.to_string_lossy().to_string();

        if config.remove_track_prefix {
            let re = TRACK_PREFIX_RE.get_or_init(|| Regex::new(r"^\d+[\.\-\s]+").unwrap());
            stem_str = re.replace(&stem_str, "").to_string();
        }

        if config.remove_source_prefix {
            let re = SOURCE_PREFIX_RE.get_or_init(|| Regex::new(r"^\s*\[.*?\]\s*").unwrap());
            stem_str = re.replace(&stem_str, "").to_string();
        }

        cleaned_name = format!("{}", stem_str.trim(), ext);
    }

    if cleaned_name != original_name {
        return RenamePreview {
            original_path: original_path_str,
            original_name,
            new_name: cleaned_name,
            status: "rules".to_string(),
            error: None,
        };
    }
}

RenamePreview {
    original_path: original_path_str,
    original_name: original_name.clone(),
    new_name: original_name,
    status: "skipped".to_string(),
    error: Some("No rules matched or missing tags".to_string()),
}
}

pub fn preview_rename(root_path: String, config: RenameConfig) -> Result<Vec<RenamePreview>, String>
{
    let validated_root = path_validator::validate_path(&root_path, None)?;
    let root_path = validated_root.to_string_lossy().to_string();
    let mut results = Vec::new();

```

```

    for entry in WalkDir::new(root_path)
        .max_depth(1)
        .into_iter()
        .filter_map(|e| e.ok())
    {
        let path = entry.path();
        if path.is_file() {
            if let Some(ext) = path.extension() {
                let ext = ext.to_string_lossy().to_lowercase();
                if is_supported_library_extension(&ext) {
                    results.push(process_file(path, &config));
                }
            }
        }
    }
}

results.sort_by(|a, b| {
    let a_changed = a.status != "skipped";
    let b_changed = b.status != "skipped";
    if a_changed && !b_changed {
        std::cmp::Ordering::Less
    } else if !a_changed && b_changed {
        std::cmp::Ordering::Greater
    } else {
        a.original_name.cmp(&b.original_name)
    }
});

Ok(results)
}

pub fn apply_rename(operations: Vec<RenameOperation>) -> Result<u32, String> {
    let mut success_count = 0;

    for mut op in operations {
        let validated_path = path_validator::validate_path(&op.original_path, None)?;
        op.original_path = validated_path.to_string_lossy().to_string();
        op.new_name = path_validator::sanitize_filename_component(&op.new_name)?;
        let src = PathBuf::from(&op.original_path);
        if let Some(parent) = src.parent() {
            let dest = parent.join(&op.new_name);
            if fs::rename(&src, &dest).is_ok() {
                success_count += 1;
            }
        }
    }

    Ok(success_count)
}

```

```

pub fn refresh_folder_songs(
    conn: Arc<Mutex<rusqlite::Connection>>,
    folder_path: String,
    minimum_duration_seconds: Option<u32>,
    cover_cache_dir: Option<PathBuf>,
) -> Result<Vec<crate::music::types::Song>, String> {
    let validated = path_validator::validate_path(&folder_path, None)?;
    let folder_path = validated.to_string_lossy().to_string();
    crate::music::scanner::scan_single_directory_internal(
        folder_path,
        conn,
        None,
        None,
        1,
        1,
        crate::music::scanner::ScanOptions::from_minimum_duration_seconds(minimum_duration_seconds),
        cover_cache_dir,
    )
}

pub fn file_exists(path: String) -> bool {
    if path_validator::validate_path(&path, None).is_err() {
        return false;
    }
    std::path::Path::new(&path).is_file()
}

pub fn resolve_download_path(
    directory: String,
    file_name: String,
    overwrite_existing: bool,
) -> Result<String, String> {
    let dir = path_validator::validate_path(&directory, None)?;
    let file_name = path_validator::sanitize_filename_component(&file_name)?;
    let direct = dir.join(&file_name);

    if overwrite_existing || !direct.exists() {
        std::fs::create_dir_all(&dir).map_err(|e| format!("创建下载目录失败: {e}"))?;
        return Ok(direct.to_string_lossy().to_string());
    }

    let dot = file_name.rfind('.');
    let (stem, ext) = match dot {
        Some(idx) => (&file_name[..idx], &file_name[idx..]),
        None => (file_name.as_str(), ""),
    };

    for i in 1..1000 {
        let candidate_name = format!("{stem} ({i}){ext}");
        let candidate = dir.join(&candidate_name);
    }
}

```

```

        if !candidate.exists() {
            return Ok(candidate.to_string_lossy().to_string());
        }
    }

    Ok(direct.to_string_lossy().to_string())
}

fn sanitize_download_filename(name: &str) -> String {
    let sanitized: String = name
        .chars()
        .map(|c| {
            if c.is_control() || matches!(c, '<' | '>' | ':' | '"' | '/' | '\\\' | '|' | '?' | '*') {
                ,
            } else {
                c
            }
        })
        .collect();
    let collapsed: String = sanitized.split_whitespace().collect::<Vec<_>>().join(" ");
    let trimmed = collapsed.trim();
    if trimmed.is_empty() {
        return "download".to_string();
    }
    trimmed.chars().take(180).collect()
}

fn ext_from_url(url: &str) -> String {
    let path = match request::Url::parse(url) {
        Ok(u) => u.path().to_string(),
        Err(_) => return String::new(),
    };
    let dot = match path.rfind('.') {
        Some(idx) => idx,
        None => return String::new(),
    };
    let ext = path[dot..].to_lowercase();
    match ext.as_str() {
        ".mp3" | ".flac" | ".wav" | ".m4a" | ".aac" | ".ape" | ".ogg" | ".wma" => ext,
        _ => String::new(),
    }
}

fn is_lossless_quality(quality: &str) -> bool {
    matches!(quality, "flac" | "flac24bit" | "hires" | "vinyl" | "master")
}

fn ext_from_quality(quality: &str) -> String {
    if is_lossless_quality(quality) {
        ".flac".to_string()
    }
}

```

```

    } else {
        ".mp3".to_string()
    }
}

fn build_filename_base(title: &str, artist: &str, album: &str, style: &str) -> String {
    let title = if title.is_empty() {
        "未知歌曲"
    } else {
        title
    };
    let parts: Vec<&str> = match style {
        "title-artist" => vec![title, artist],
        "title-artist-album" => vec![title, artist, album],
        _ => vec![artist, title],
    };
    let joined: String = parts
        .iter()
        .map(|p| p.trim().to_string())
        .filter(|p| !p.is_empty())
        .collect::<Vec<_>>()
        .join(" - ");
    if joined.is_empty() {
        title.to_string()
    } else {
        joined
    }
}

fn build_download_filename(
    title: &str,
    artist: &str,
    album: &str,
    url: &str,
    quality: &str,
    keep_source_filename: bool,
    style: &str,
) -> String {
    let ext = {
        let e = ext_from_url(url);
        if e.is_empty() {
            ext_from_quality(quality)
        } else {
            e
        }
    };

    if keep_source_filename {
        if let Ok(u) = request::Url::parse(url) {
            let path = u.path();

```

```

        if let Some(base) = path.rsplitt('/').next() {
            if let Some(dot_idx) = base.rfind('.') {
                let stem = &base[..dot_idx];
                let decoded = urlencoding::decode(stem)
                    .map(|cow| cow.into_owned())
                    .unwrap_or_else(|_| stem.to_string());
                if !decoded.is_empty() {
                    return format!("{}", sanitize_download_filename(&decoded), ext);
                }
            }
        }
    }
}

```

```

let base = build_filename_base(title, artist, album, style);
format!("{}", sanitize_download_filename(&base), ext)
}

```

```

pub fn resolve_download_full_path(
    directory: String,
    title: String,
    artist: String,
    album: String,
    url: String,
    quality: String,
    keep_source_filename: bool,
    file_name_style: String,
    overwrite_existing: bool,
) -> Result<String, String> {
    let validated_dir = path_validator::validate_path(&directory, None)?;
    let directory = validated_dir.to_string_lossy().to_string();
    let file_name = build_download_filename(
        &title,
        &artist,
        &album,
        &url,
        &quality,
        keep_source_filename,
        &file_name_style,
    );
    let file_name = path_validator::sanitize_filename_component(&file_name)?;
    resolve_download_path(directory, file_name, overwrite_existing)
}

```

```

pub fn build_download_basename(
    title: String,
    artist: String,
    album: String,
    file_name_style: String,
) -> String {

```

```

    let base = build_filename_base(&title, &artist, &album, &file_name_style);
    let cleaned = sanitize_download_filename(&base);
    path_validator::sanitize_filename_component(&cleaned).unwrap_or_else(|_| "download".to_string())
}

```

```

const DOWNLOAD_HISTORY_FILE: &str = "download_history.json";

```

```

pub async fn check_update_by_rust(owner: String, repo: String) -> Result<String, String> {
    let url = format!("https://api.github.com/repos/{owner}/{repo}/releases/latest");

```

```

    let client = reqwest::Client::builder()
        .timeout(std::time::Duration::from_secs(10))
        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("XY-Music-Updater")
        .build()
        .map_err(|e| format!("创建更新请求失败: {e}"))?;

```

```

    client
        .get(&url)
        .header("Accept", "application/vnd.github+json")
        .send()
        .await
        .map_err(|e| format!("请求更新接口失败: {e}"))?
        .error_for_status()
        .map_err(|e| format!("更新接口返回错误状态: {e}"))?
        .text()
        .await
        .map_err(|e| format!("读取更新数据失败: {e}"))
}

```

```

#[derive(Debug, Clone, serde::Serialize)]

```

```

pub struct SongDownloadProgress {
    pub progress: f64,
    pub downloaded: u64,
    pub total: u64,
    pub speed: f64,
}

```

```

pub async fn download_online_song(
    url: String,
    dest_path: String,
    ekey: Option<String>,
    cek: Option<String>,
    headers: Option<std::collections::HashMap<String, String>>,
) -> Result<String, String> {
    use std::time::Instant;
    use tokio::fs::File;
    use tokio::io::AsyncWriteExt;

    if !(url.starts_with("http://") || url.starts_with("https://")) {

```

```

        return Err("无效的下载链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("下载链接校验失败: {e}"))?;

    let client = request::Client::builder()
        .timeout(std::time::Duration::from_secs(600))
        .redirect(crate::security::ssrf::ssrf_redirect_policy())
        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
        .build()
        .map_err(|e| format!("创建下载请求客户端失败: {e}"))?;

    let send_with_range = |with_range: bool| {
        let mut builder = client.get(&url).header(
            "Accept",
            "audio/webm, audio/ogg, audio/wav, audio/*;q=0.9, application/ogg;q=0.7, video/*;q=0.6, */*;q=0.5",
        );
        if with_range {
            builder = builder.header("Range", "bytes=0-");
        }
        if let Some(ref hdrs) = headers {
            for (key, value) in hdrs {
                if key.eq_ignore_ascii_case("accept") || key.eq_ignore_ascii_case("range") {
                    continue;
                }
                builder = builder.header(key.as_str(), value.as_str());
            }
        }
        builder.send()
    };

    let mut response = send_with_range(true)
        .await
        .map_err(|e| format!("发送下载请求失败: {e}"))?;
    let status_code = response.status().as_u16();
    if !response.status().is_success()
        && (status_code == 502 || status_code == 416 || status_code == 403)
    {
        response = send_with_range(false)
            .await
            .map_err(|e| format!("发送下载请求 (无 Range 回退) 失败: {e}"))?;
    }
    if !response.status().is_success() {
        return Err(format!("下载服务器返回错误状态: {}", response.status()));
    }

    let total_size = response.content_length().unwrap_or(0);

```

```

let dest = PathBuf::from(&dest_path);
if let Some(parent) = dest.parent() {
    tokio::fs::create_dir_all(parent)
        .await
        .map_err(|e| format!("创建下载目录失败: {e}"))?;
}

let mut file = File::create(&dest)
    .await
    .map_err(|e| format!("创建目标文件失败: {e}"))?;
let mut downloaded: u64 = 0;
let start_time = Instant::now();

let mut response = response;
loop {
    match response.chunk().await {
        Ok(Some(chunk)) => {
            file.write_all(&chunk)
                .await
                .map_err(|e| format!("写入文件失败: {e}"))?;
            downloaded += chunk.len() as u64;
        }
        Ok(None) => break,
        Err(e) => {
            drop(file);
            let _ = tokio::fs::remove_file(&dest).await;
            return Err(format!("下载数据分块失败: {e}"));
        }
    }
}

file.flush()
    .await
    .map_err(|e| format!("刷新文件缓存失败: {e}"))?;
drop(file);

if total_size > 0 && downloaded < total_size {
    let _ = tokio::fs::remove_file(&dest).await;
    return Err(format!(
        "下载不完整 ({downloaded}/{total_size} 字节), 可能被其他下载器 (如 IDM) 拦截。请在下载器设置中排除本应用, 或临时退出下载器后重试。"
    ));
}

if let Some(ref ek) = ekey {
    if !ek.is_empty() {
        let dest_clone = dest.clone();
        let ek_clone = ek.clone();
        let decrypted = tokio::task::spawn_blocking(move || {
            decrypt_qmc_file_inplace(&dest_clone, &ek_clone)
        })
    }
}

```

```

    ))
    .await
    .map_err(|e| format!("解密任务执行失败: {e}"))?;
    decrypted.map_err(|e| format!("QMC2 解密失败: {e}"))?;
}
} else if let Some(ref ck) = cek {
    if !ck.is_empty() {

        let dest_clone = dest.clone();
        let ck_clone = ck.clone();
        let decrypted = tokio::task::spawn_blocking(move || {
            crate::player::stream_cache::decrypt_cenc_file(&dest_clone, &ck_clone)
        })
        .await
        .map_err(|e| format!("解密任务执行失败: {e}"))?;
        decrypted.map_err(|e| format!("CENC 解密失败: {e}"))?;
    }
} else if let Some(extracted_ekey) = try_extract_ekey_from_file(&dest) {
    let dest_clone = dest.clone();
    let decrypted = tokio::task::spawn_blocking(move || {
        decrypt_qmc_file_inplace(&dest_clone, &extracted_ekey)
    })
    .await
    .map_err(|e| format!("解密任务执行失败: {e}"))?;
    decrypted.map_err(|e| format!("QMC2 解密失败 (footer ekey) : {e}"))?;
}

let _ = start_time;
Ok(dest.to_string_lossy().to_string())
}

fn try_extract_ekey_from_file(path: &Path) -> Option<String> {
    let metadata = fs::metadata(path).ok()?;
    let file_size = metadata.len();
    if file_size < 8 {
        return None;
    }

    let tail_size = (file_size.min(4096)) as usize;
    let mut file = fs::File::open(path).ok()?;
    use std::io::{Read, Seek, SeekFrom};
    file.seek(SeekFrom::Start(file_size - tail_size as u64)).ok()?;
    let mut tail = vec![0u8; tail_size];
    file.read_exact(&mut tail).ok()?;

    crate::player::qmc2::extract_ekey_from_footer(&tail)
}

fn decrypt_qmc_file_inplace(path: &Path, ekey: &str) -> Result<u64, String> {
    let crypto = crate::player::qmc2::QmcCrypto::from_ekey(ekey)

```

```

        .map_err(|e| format!("ekey 解析失败: {e}"))?;
        decrypt_with_crypto_inplace(path, &crypto)
    }
}

fn decrypt_with_crypto_inplace(path: &Path, crypto: &crate::player::qmc2::QmcCrypto) -> Result<u64,
String> {
    use std::io::{Read, Write};

    let file_size = fs::metadata(path)
        .map_err(|e| format!("读取文件元数据失败: {e}"))?
        .len();

    let temp_path = path.with_extension("qmc_tmp_dec");

    let write_result: Result<(), String> = (|| {
        let mut input = fs::File::open(path)
            .map_err(|e| format!("打开加密文件失败: {e}"))?;
        let mut output = fs::File::create(&temp_path)
            .map_err(|e| format!("创建临时解密文件失败: {e}"))?;

        let mut offset: u64 = 0;
        let mut buf = vec![0u8; 64 * 1024];

        loop {
            let n = input.read(&mut buf)
                .map_err(|e| format!("读取加密数据失败: {e}"))?;
            if n == 0 {
                break;
            }
            crypto.decrypt(offset as usize, &mut buf[..n]);
            output.write_all(&buf[..n])
                .map_err(|e| format!("写入解密数据失败: {e}"))?;
            offset += n as u64;
        }

        output.flush()
            .map_err(|e| format!("刷新解密文件失败: {e}"))
    })();

    if let Err(e) = write_result {
        let _ = fs::remove_file(&temp_path);
        return Err(e);
    }

    fs::rename(&temp_path, path)
        .map_err(|e| {
            let _ = fs::remove_file(&temp_path);
            format!("替换原文件失败: {e}")
        })?;

    Ok(file_size)
}

```

```

}

fn read_file_header(path: &Path) -> Result<Vec<u8>, String> {
    use std::io::Read;
    let mut file = fs::File::open(path).map_err(|e| format!("打开文件失败: {e}"))?;
    let mut header = vec![0u8; 16];
    let mut filled = 0;
    while filled < header.len() {
        let n = file.read(&mut header[filled..]).map_err(|e| format!("读取文件头失败: {e}"))?;
        if n == 0 {
            break;
        }
        filled += n;
    }
    header.truncate(filled);
    Ok(header)
}

```

```

fn is_qmcl_extension(path: &Path) -> bool {
    let Some(name) = path.file_name().and_then(|n| n.to_str()) else {
        return false;
    };
    let lower = name.to_lowercase();
    matches!(
        lower.as_str(),
        "qmcflac" | "qmcmp3" | "qmcogg" | "qmcwav" | "qmcape" | "qmcwma"
    ) || lower.starts_with("qmc")
}

```

```

fn rename_to_audio_extension(path: &Path) -> Result<Option<PathBuf>, String> {
    let header = read_file_header(path)?;
    let Some(real_ext) = crate::player::qmc2::detect_audio_extension(&header) else {
        return Ok(None);
    };
    let cur_ext = path
        .extension()
        .and_then(|e| e.to_str())
        .map(|e| e.to_lowercase())
        .unwrap_or_default();
    if cur_ext == real_ext {
        return Ok(None);
    }
    let stem = path.file_stem().unwrap_or_default();
    let new_path = path.with_file_name(format!("{}", stem.to_string_lossy(), real_ext));
    if new_path.exists() && new_path != path {
        return Ok(None);
    }
    fs::rename(path, &new_path).map_err(|e| format!("修正扩展名失败: {e}"))?;
    Ok(Some(new_path))
}

```

```

pub fn decrypt_qmc_file_standalone(
    file_path: String,
    ekey: Option<String>,
) -> Result<String, String> {
    let path = path_validator::validate_path(&file_path, None)?;

    if !path.is_file() {
        return Err(format!("文件不存在: {}", path.display()));
    }

    let actual_ekey = if let Some(ref ek) = ekey {
        if !ek.is_empty() {
            Some(ek.clone())
        } else {
            try_extract_ekey_from_file(&path)
        }
    } else {
        try_extract_ekey_from_file(&path)
    };

    let (decrypt_result, crypto_name) = if let Some(ek) = actual_ekey {
        let crypto = crate::player::qmc2::QmcCrypto::from_ekey(&ek)
            .map_err(|e| format!("ekey 解析失败: {e}"))?;
        (decrypt_with_crypto_inplace(&path, &crypto), "QMC2")
    } else if is_qmc1_extension(&path) {
        let crypto = crate::player::qmc2::QmcCrypto::qmc1();
        let backup = path.with_extension("qmc_tmp_backup");
        fs::copy(&path, &backup).map_err(|e| format!("备份原文件失败: {e}"))?;
        let result = decrypt_with_crypto_inplace(&path, &crypto);
        match result {
            Ok(_) => {
                let header = read_file_header(&path).unwrap_or_default();
                if crate::player::qmc2::detect_audio_extension(&header).is_none() {
                    let _ = fs::rename(&backup, &path);
                    return Err("解密结果不是有效音频（该文件可能不是 QMC1 格式）".to_string());
                }
                let _ = fs::remove_file(&backup);
                (Ok(0), "QMC1")
            }
            Err(e) => {
                let _ = fs::rename(&backup, &path);
                return Err(format!("QMC1 解密失败: {e}"));
            }
        }
    } else {
        return Ok(serde_json::json!({
            "status": "not_encrypted",
            "outputPath": path.to_string_lossy(),
            "renamedTo": null,
        }));
    }
}

```

```

    })
    .to_string());
};

match decrypt_result {
    Ok(_) => {
        let renamed_to = rename_to_audio_extension(&path)?;
        let output = renamed_to.as_ref().unwrap_or(&path);
        Ok(serde_json::json!({
            "status": "decrypted",
            "outputPath": output.to_string_lossy(),
            "renamedTo": renamed_to.as_ref().map(|p| p.to_string_lossy().to_string()),
            "crypto": crypto_name,
        }))
        .to_string())
    }
    Err(e) => Err(format!("{} 解密失败: {}", e)),
}

}

pub fn decrypt_qmc_file(file_path: String, ekey: Option<String>) -> Result<bool, String> {
    let path = path_validator::validate_path(&file_path, None)?;

    if !path.is_file() {
        return Err(format!("文件不存在: {}", path.display()));
    }

    let actual_ekey = if let Some(ref ek) = ekey {
        if !ek.is_empty() {
            Some(ek.clone())
        } else {
            try_extract_ekey_from_file(&path)
        }
    } else {
        try_extract_ekey_from_file(&path)
    };

    if let Some(ek) = actual_ekey {
        match decrypt_qmc_file_inplace(&path, &ek) {
            Ok(_) => Ok(true),
            Err(e) => Err(format!("QMC2 解密失败: {}", e)),
        }
    } else {
        Ok(false)
    }
}

pub async fn save_download_lyrics(content: String, dest_path: String) -> Result<String, String> {
    write_text_file(content, dest_path).await
}

```

```

pub async fn write_text_file(content: String, dest_path: String) -> Result<String, String> {
    let dest = path_validator::validate_path(&dest_path, None)?;
    if let Some(parent) = dest.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建目录失败: {e}"))?;
    }
    tokio::fs::write(&dest, content)
        .await
        .map_err(|e| format!("写入文件失败: {e}"))?;
    Ok(dest.to_string_lossy().to_string())
}

#[derive(Debug, Serialize)]
pub struct FetchedImage {
    pub data: Vec<u8>,
    pub mime: String,
}

pub async fn fetch_image_bytes(url: String) -> Result<FetchedImage, String> {
    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的图片链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("图片链接校验失败: {e}"))?;

    let client = reqwest::Client::builder()
        .timeout(std::time::Duration::from_secs(30))
        .redirect(crate::security::ssrf::ssrf_redirect_policy())
        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
        .build()
        .map_err(|e| format!("创建请求客户端失败: {e}"))?;

    let response = client
        .get(&url)
        .send()
        .await
        .map_err(|e| format!("请求图片失败: {e}"))?;

    if !response.status().is_success() {
        return Err(format!("图片服务器返回错误状态: {}", response.status()));
    }

    let mime = response
        .headers()
        .get(reqwest::header::CONTENT_TYPE)

```

```

        .and_then(|v| v.to_str().ok())
        .unwrap_or("image/jpeg")
        .to_string();

let data = response
    .bytes()
    .await
    .map_err(|e| format!("读取图片数据失败: {e}"))?
    .to_vec();

if data.is_empty() {
    return Err("图片数据为空".to_string());
}

Ok(FetchedImage { data, mime })
}

pub async fn save_download_bytes(data: Vec<u8>, dest_path: String) -> Result<String, String> {
    if data.is_empty() {
        return Err("下载数据为空".to_string());
    }

    let dest = path_validator::validate_path(&dest_path, None)?;
    if let Some(parent) = dest.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建下载目录失败: {e}"))?;
    }

    tokio::fs::write(&dest, &data)
        .await
        .map_err(|e| format!("写入文件失败: {e}"))?;
    Ok(dest.to_string_lossy().to_string())
}

pub async fn embed_audio_metadata(request: EmbedMetadataRequest) -> Result<(), String> {
    let request = request.clone();
    tokio::task::spawn_blocking(move || write_metadata_to_file(&request))
        .await
        .map_err(|e| format!("元数据嵌入任务失败: {e}"))?
}

#[derive(Debug, serde::Deserialize, Default)]
#[serde(rename_all = "camelCase")]
pub struct FinalizeDownloadExtrasRequest {
    pub lyrics_text: Option<String>,
    pub lyrics_path: Option<String>,
    pub cover_url: Option<String>,
    pub cover_path: Option<String>,
    pub metadata: Option<EmbedMetadataRequest>,
    pub embed_cover: bool,
}

```

```

#[derive(Debug, Serialize, Default)]
pub struct FinalizeDownloadExtrasResult {
    pub lyrics_saved: bool,
    pub cover_saved: bool,
    pub metadata_embedded: bool,
    pub metadata_error: Option<String>,
    pub cover_data: Option<Vec<u8>>,
    pub cover_mime: String,
}

pub async fn finalize_download_extras(
    request: FinalizeDownloadExtrasRequest,
) -> Result<FinalizeDownloadExtrasResult, String> {
    let mut result = FinalizeDownloadExtrasResult::default();

    if let (Some(text), Some(path)) = (&request.lyrics_text, &request.lyrics_path) {
        if !text.is_empty() {
            let dest = PathBuf::from(path);
            if let Some(parent) = dest.parent() {
                let _ = tokio::fs::create_dir_all(parent).await;
            }
            match tokio::fs::write(&dest, text).await {
                Ok(_) => result.lyrics_saved = true,
                Err(_) => {}
            }
        }
    }

    if let Some(url) = &request.cover_url {
        if !url.is_empty() && (url.starts_with("http://") || url.starts_with("https://")) {
            match fetch_image_bytes(url.clone()).await {
                Ok(img) => {
                    if let Some(path) = &request.cover_path {
                        let actual_ext = if img.mime.contains("png") {
                            ".png"
                        } else {
                            ".jpg"
                        };
                        let final_path = if path.ends_with(".jpg") && actual_ext == ".png" {
                            format!("{}", path[..path.len() - 4])
                        } else if path.ends_with(".png") && actual_ext == ".jpg" {
                            format!("{}", path[..path.len() - 4])
                        } else {
                            path.clone()
                        };
                        let dest = PathBuf::from(&final_path);
                        if let Some(parent) = dest.parent() {
                            let _ = tokio::fs::create_dir_all(parent).await;
                        }
                    }
                }
            }
        }
    }
}

```

```

        match tokio::fs::write(&dest, &img.data).await {
            Ok(_) => {
                result.cover_saved = true;
            }
            Err(_) => {}
        }
    }
    result.cover_data = Some(img.data);
    result.cover_mime = img.mime;
}
Err(_) => {}
}

}

if let Some(mut meta) = request.metadata {
    if request.embed_cover && meta.cover_data.is_none() {
        if let Some(data) = &result.cover_data {
            meta.cover_data = Some(data.clone());
            meta.cover_mime = Some(result.cover_mime.clone());
        }
    }
    let meta = meta.clone();
    match tokio::task::spawn_blocking(move || write_metadata_to_file(&meta)).await {
        Ok(Ok(())) => result.metadata_embedded = true,
        Ok(Err(e)) => {
            result.metadata_error = Some(e);
        }
        Err(e) => {
            result.metadata_error = Some(format!("元数据嵌入任务失败: {e}"));
        }
    }
}

Ok(result)
}

fn download_history_path(data_dir: &Path) -> Result<PathBuf, String> {
    Ok(data_dir.join(DOWNLOAD_HISTORY_FILE))
}

pub async fn read_download_history(data_dir: &Path) -> Result<String, String> {
    let path = download_history_path(data_dir)?;
    if !path.is_file() {
        return Ok("{}".to_string());
    }
    match tokio::fs::read_to_string(&path).await {
        Ok(content) if !content.trim().is_empty() => Ok(content),
        Ok(_) => Ok("{}".to_string()),
        Err(e) => Err(format!("读取下载记录失败: {e}")),
    }
}

```

```

    }
}

pub async fn write_download_history(data_dir: &Path, content: String) -> Result<(), String> {
    let path = download_history_path(data_dir)?;
    if let Some(parent) = path.parent() {
        tokio::fs::create_dir_all(parent)
            .await
            .map_err(|e| format!("创建下载记录目录失败: {e}"))?;
    }
    tokio::fs::write(&path, content)
        .await
        .map_err(|e| format!("写入下载记录失败: {e}"))?;
    Ok(())
}

#[derive(Debug, Clone, serde::Serialize)]
pub struct ProbeUrlInfo {
    pub url: String,
    pub size: u64,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub error: Option<String>,
}

pub async fn probe_url_size(url: String) -> Result<ProbeUrlInfo, String> {
    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的探测链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("探测链接校验失败: {e}"))?;

    let client = request::Client::builder()
        .timeout(std::time::Duration::from_secs(8))
        .redirect(crate::security::ssrf::ssrf_redirect_policy())
        .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
        .user_agent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
        .build()
        .map_err(|e| format!("创建探测客户端失败: {e}"))?;

    let resp = match client
        .get(&url)
        .header(request::header::RANGE, "bytes=0-0")
        .header(
            request::header::ACCEPT,
            "audio/webm, audio/ogg, audio/wav, audio/*;q=0.9, */*;q=0.5",
        )
        .send()
        .await

```

```

    {
        Ok(r) => r,
        Err(e) => {
            return Ok(ProbeUrlInfo {
                url,
                size: 0,
                error: Some(format!("请求失败: {e}")),
            });
        }
    };

    let final_url = resp.url().to_string();
    let status = resp.status();

    if status == request::StatusCode::PARTIAL_CONTENT {
        let size = resp
            .headers()
            .get(request::header::CONTENT_RANGE)
            .and_then(|v| v.to_str().ok())
            .and_then(|v| v.rsplit('/').next())
            .and_then(|v| v.parse::<u64>().ok())
            .unwrap_or(0);
        return Ok(ProbeUrlInfo {
            url: final_url,
            size,
            error: None,
        });
    }

    if status.is_success() {
        let size = resp
            .headers()
            .get(request::header::CONTENT_LENGTH)
            .and_then(|v| v.to_str().ok())
            .and_then(|v| v.trim().parse::<u64>().ok())
            .unwrap_or(0);
        return Ok(ProbeUrlInfo {
            url: final_url,
            size,
            error: None,
        });
    }

    Ok(ProbeUrlInfo {
        url: final_url,
        size: 0,
        error: Some(format!("HTTP {status}")),
    })
}

pub async fn fetch_announcement() -> Result<String, String> {

```

```

let url = "https://xy.zh2026.cn/chaoguan/public/api/app.php?action=app_announcement";

let client = request::Client::builder()
    .timeout(std::time::Duration::from_secs(15))
    .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
    .user_agent("XY-Music-Updater")
    .http1_only()
    .build()
    .map_err(|e| format!("创建请求客户端失败: {e}"))?;

let resp = client
    .get(url)
    .header("Accept", "application/json")
    .header("Cache-Control", "no-cache")
    .send()
    .await
    .map_err(|e| format!("请求公告接口失败: {e}"))?;

let status = resp.status();
let text = resp
    .text()
    .await
    .map_err(|e| format!("读取公告数据失败: {e}"))?;

if !status.is_success() {
    let snippet: String = text.chars().take(200).collect();
    return Err(format!("公告接口返回错误状态: {status} | 响应: {snippet}"));
}

match serde_json::from_str::(<serde_json::Value>(&text)) {
    Ok(v) => {
        let code = v.get("code").and_then(|c| c.as_i64()).unwrap_or(0);
        if code == 200 {
            match v.get("data") {
                Some(d) if !d.is_null() => return Ok(d.to_string()),
                _ => return Ok("{}".to_string()),
            }
        }
        let msg = v
            .get("msg")
            .and_then(|m| m.as_str())
            .unwrap_or("公告接口返回未知错误");
        Err(format!("公告接口返回错误: {msg}"))
    }
    Err(_) => {
        let snippet: String = text.chars().take(200).collect();
        Err(format!("公告数据解析失败, 原始响应: {snippet}"))
    }
}
}

```

```
pub async fn write_state_json(data_dir: &Path, key: String, value: String) -> Result<(), String> {
    let sanitized_key = path_validator::sanitize_filename_component(&key)
        .map_err(|e| format!("无效的 key: {}", e))?;
    let state_dir = data_dir.join("state");
    tokio::fs::create_dir_all(&state_dir)
        .await
        .map_err(|e| format!("创建 state 目录失败: {e}")?);
    let file_path = state_dir.join(format!("{}", sanitized_key).json());
    tokio::fs::write(&file_path, &value)
        .await
        .map_err(|e| format!("写入 state 文件失败: {e}")?);
    Ok(())
}
```

```
pub async fn read_state_json(data_dir: &Path, key: String) -> Result<Option<String>, String> {
    let sanitized_key = path_validator::sanitize_filename_component(&key)
        .map_err(|e| format!("无效的 key: {}", e))?;
    let file_path = data_dir.join("state").join(format!("{}", sanitized_key).json());
    if !file_path.exists() {
        return Ok(None);
    }
    let content = tokio::fs::read_to_string(&file_path)
        .await
        .map_err(|e| format!("读取 state 文件失败: {e}")?);
    Ok(Some(content))
}
```

```
pub async fn download_wallpaper(
    data_dir: &Path,
    url: String,
    filename: String,
) -> Result<String, String> {
    use tokio::fs::File;
    use tokio::io::AsyncWriteExt;

    if !(url.starts_with("http://") || url.starts_with("https://")) {
        return Err("无效的壁纸下载链接".to_string());
    }

    crate::security::ssrf::validate_outbound_url(&url)
        .await
        .map_err(|e| format!("壁纸链接校验失败: {e}")?);

    let safe_name = std::path::Path::new(&filename)
        .file_name()
        .and_then(|n| n.to_str())
        .unwrap_or("wallpaper.jpg")
        .to_string();
    let safe_name = if std::path::Path::new(&safe_name).extension().is_none() {
```

```

        format!("{}", safe_name).jpg")
    } else {
        safe_name
    };

let wallpaper_dir = data_dir.join("wallpapers");
tokio::fs::create_dir_all(&wallpaper_dir)
    .await
    .map_err(|e| format!("创建壁纸目录失败: {e}"))?;
let dest_path = wallpaper_dir.join(&safe_name);

let client = reqwest::Client::builder()
    .timeout(std::time::Duration::from_secs(60))
    .redirect(crate::security::ssrf::ssrf_redirect_policy())
    .dns_resolver(crate::security::ssrf::pinned_dns_resolver())
    .user_agent("XY-Music-WallpaperDownloader")
    .build()
    .map_err(|e| format!("创建HTTP客户端失败: {e}"))?;

let mut response = client
    .get(&url)
    .send()
    .await
    .map_err(|e| format!("下载壁纸失败: {e}"))?;
if !response.status().is_success() {
    return Err(format!("下载服务器返回错误状态: {}", response.status()));
}

let mut file = File::create(&dest_path)
    .await
    .map_err(|e| format!("创建文件失败: {e}"))?;
while let Some(chunk) = response
    .chunk()
    .await
    .map_err(|e| format!("读取响应数据失败: {e}"))?
{
    file.write_all(&chunk)
        .await
        .map_err(|e| format!("写入文件失败: {e}"))?;
}

Ok(dest_path.to_string_lossy().to_string())
}

pub async fn delete_wallpaper_file(data_dir: &Path, local_path: String) -> Result<(), String> {
    let wallpaper_dir = data_dir.join("wallpapers");
    let target = PathBuf::from(&local_path);

    if !target.exists() {
        return Ok(());
    }

```

```

    }

    if !target.is_file() {
        return Err("目标不是可删除的壁纸文件".to_string());
    }

    let canonical_dir = std::fs::canonicalize(&wallpaper_dir)
        .map_err(|e| format!("读取壁纸目录失败: {e}"))?;
    let canonical_target = std::fs::canonicalize(&target)
        .map_err(|e| format!("读取壁纸文件失败: {e}"))?;
    if !canonical_target.starts_with(&canonical_dir) {
        return Err("只能删除应用壁纸目录中的文件".to_string());
    }

    tokio::fs::remove_file(&target)
        .await
        .map_err(|e| format!("删除壁纸文件失败: {e}"))?;
    Ok(())
}

#[cfg(test)]
mod tests {
    use super::FinalizeDownloadExtrasRequest;

    #[test]
    fn finalize_download_extras_request_accepts_frontend_camel_case_payload() {
        let json = serde_json::json!({
            "lyricsText": "[00:00.00]测试歌词",
            "lyricsPath": "/Music/song.lrc",
            "coverUrl": "https://example.com/cover.jpg",
            "coverPath": "/Music/song.jpg",
            "embedCover": true,
            "metadata": {
                "filePath": "/Music/song.mp3",
                "title": "测试歌曲",
                "albumArtist": "测试专辑艺术家",
                "trackNumber": "7",
                "coverMime": "image/jpeg"
            }
        });

        let request: FinalizeDownloadExtrasRequest =
            serde_json::from_value(json).expect("frontend payload should deserialize");

        assert_eq!(request.lyrics_text.as_deref(), Some("[00:00.00]测试歌词"));
        assert_eq!(request.lyrics_path.as_deref(), Some("/Music/song.lrc"));
        assert_eq!(request.cover_url.as_deref(), Some("https://example.com/cover.jpg"));
        assert_eq!(request.cover_path.as_deref(), Some("/Music/song.jpg"));
        assert!(request.embed_cover);

        let metadata = request.metadata.expect("metadata should deserialize");
        assert_eq!(metadata.file_path, "/Music/song.mp3");
        assert_eq!(metadata.title.as_deref(), Some("测试歌曲"));
    }
}

```

```
        assert_eq!(metadata.album_artist.as_deref(), Some("测试专辑艺术家"));
        assert_eq!(metadata.track_number.as_deref(), Some("7"));
        assert_eq!(metadata.cover_mime.as_deref(), Some("image/jpeg"));
    }
}
```